

Autoregressive Inference Before Tools: The Multi-Token Computational Frontier of Tiny Transformers

Paper 0.85 — Multi-Token Inference Research Line

Stage-1 results — August 2026

Abstract

A one-token Transformer is a fixed-depth circuit, but a decoder generating multiple tokens is already an iterative machine: each generated token becomes writable state, and subsequent steps can attend to the full trajectory. Before attributing systematic reasoning gains to external agents or tools, we measure how much computation ordinary autoregressive generation learns by itself. A leakage-audited local gate first establishes one-step implication over held-out ordered pairs with shared, fully trained lexical support: all three L2/W64/H4 seeds exceed 98.8% on positive and negative controls. We then train proof chains only at $K \leq 3$ and test $K \in \{1, 2, 3, 4, 6, 8\}$. One-token answers (O0), free traces (O1), and fully teacher-forced structured traces (O3) are perfect in range but fail abruptly at $K = 4$ for every seed. Thus $K_{\text{internal}} = K_{\text{AR}} = K_{\text{O3}} = 3$ and measured recurrence gain is zero. Structured grammar/final supervision without intermediate state labels (O2) learns formatting but no exact trajectories. An earlier fully disjoint-symbol run is retained only as a lexical-transfer stress, and a query-target leakage pilot is quarantined. The result is a stable finite train-short/test-long boundary, not a closure signature, and supplies the model-only baseline for Paper 0.9.

1 Motivation

Suppose the context contains

$$A, \quad A \Rightarrow B, \quad B \Rightarrow C, \quad C \Rightarrow D.$$

A one-token answer to query D ? requires internal composition before output. An autoregressive model can instead emit

$$A \rightarrow B \rightarrow C \rightarrow D$$

or a structured trajectory such as

$$KNOWN(A), \text{ DERIVE}(B), \text{ DERIVE}(C), \text{ DERIVE}(D), \text{ ANSWER}(TRUE).$$

The model weights are unchanged, but the effective machine is not. Generated tokens form external state:

$$s_{t+1} = F_{\theta}(X, s_{\leq t}).$$

Paper 0.85 isolates this resource before adding tools.

2 Scope

The first version uses deliberately simple propositional implication. More complex single-step syntax—negation, conjunction, variables, predicates, functors, frames, algebraic rewrites—should only be added after Paper 0.7 establishes which local operators are reliably learned.

The primary question is therefore:

given a reliably learned local implication step, can autoregression learn reusable proof recurrence?

3 Task prerequisite

Every recurrence experiment first passes a local-step gate:

$$A, \quad A \Rightarrow B \vdash B.$$

A model that cannot reliably solve this across held-out proposition pairs cannot support an interpretable closure experiment. Report $P_{\text{acquire}}^{\text{local}}$ across model and generator seeds, while ensuring that lexical output support is not accidentally withheld.

R0-v1 withheld proposition identities entirely. Its three seeds learn the negative label (98.8–99.6%) but achieve only 4.3–6.6% entailed accuracy and 4.7–7.8% on consequent swaps despite near-zero training loss. Because held-out symbols also have untrained output rows, this is a useful lexical/OOV-output transfer stress, not the central implication gate.

R0-v2 corrects that confound. All 64 symbols occur during training as inputs and positive targets; 3,226 ordered implication pairs train the model and 806 pairs are held out, with zero observed pair leakage. Entailed, consequent-swap, fact-mismatch, and rule-LHS-mismatch controls are balanced. At L2/W64/H4, seed 11 scores 100% on all four conditions; seed 23 scores 100%, 100%, 100%, and 99.6%; seed 37 scores 98.8%, 98.8%, 99.6%, and 100%. The preregistered 95% three-seed gate therefore passes. This establishes local implication over held-out pairs with trained lexical support; it does not establish recurrence.

4 Dataset ladder

4.1 R0: direct one-step implication

Use arbitrary proposition symbols and disjoint train/test identities where feasible.

4.2 R1: fixed two-step chains

$$A, \quad A \Rightarrow B, \quad B \Rightarrow C \vdash C.$$

Train both direct-answer and derivation-output conditions.

4.3 R2: variable chain length

Generate

$$A_0, \quad A_0 \Rightarrow A_1, \dots, A_{K-1} \Rightarrow A_K.$$

Train $K \leq K_{\text{train}}$, initially $K_{\text{train}} \in \{2, 3\}$. Test

$$K \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64\}.$$

4.4 R3: shuffled rule order

Randomize implication order to remove a trivial left-to-right copying strategy.

4.5 R4: distractor rules

Add irrelevant implications with

$$N \in \{0, 2, 4, 8, 16, 32, 64\}.$$

4.6 R5: branching

Introduce multiple outgoing rules and competing branches only after chain recurrence works. Keep a unique target proof initially, then increase search ambiguity.

4.7 Executable generator and split contract

The broad CPU instrument stores the fact, ordered ground-truth chain, implication multiset, query, minimal proof depth, distractor count, branching factor, shuffle flag, split, and split-specific namespace. Branches terminate away from the query, so the target proof remains unique; distractor antecedents are unreachable from the initial fact. The exact validator checks every chain edge, namespace membership, and target-path uniqueness before serialization.

The full CPU materialization contains 2,800 examples. Training is strictly bounded to $K \in \{1, 2, 3\}$; validation and test cover all eleven preregistered depths. All splits cross seven distractor levels, ordered/shuffled presentations, and branching factors one/two. Every row passes the symbolic validator. Four exact target serializations per example produce 11,200 reference traces. These reference outputs pass exact trajectory checks and are labeled `reference_oracle`; they establish generator correctness, not model performance.

The learned Stage-1 instrument uses the R0-v2 shared vocabulary and partitions ordered implication edges rather than lexical identities. Every edge in a training chain belongs to the training pair set and every evaluation edge belongs to the held-out pair set; observed edge leakage is zero. The query is a bare instruction token and contains no answer argument. This last audit was added after an initial pilot serialized `QUERY target`, producing a trivial copy path. That pilot is quarantined as invalid and contributes no result or figure.

5 Output-machine conditions

O0: one-token answer. Emit final conclusion/boolean only.

O1: free multi-token generation. Train an untyped proposition trace and score final answer, transitions, exact trajectory, and termination.

O2: structured scratchpad without state labels. Supervise grammar/control tokens and the final answer, but withhold intermediate proposition labels. Use minimal tokens such as

$$\langle STATE : A \rangle \langle DERIVE : B \rangle \cdots \langle ANSWER : D \rangle.$$

No external semantic operation is executed.

O3: teacher-forced derivation training. Supervise every structured intermediate proof token in training, then autoregress at test. The O2–O3 contrast isolates intermediate semantic supervision from format supervision; neither condition executes an external operation.

6 Core recurrence hypothesis

A learned proof procedure should approximate

$$\begin{aligned} s_t &= \text{current known proposition,} \\ s_{t+1} &= \text{NextApplicableRule}(s_t, \mathcal{R}). \end{aligned}$$

For a pure chain this transition is local and time-invariant. If acquired, larger test proof depth should primarily increase generated steps rather than require a larger network.

7 Internal versus autoregressive composition

Define

$$K_{\text{internal}}(M)$$

as the largest one-token chain length meeting the competence gate, and

$$K_{\text{AR}}(M)$$

as the largest multi-token chain length meeting the same final-answer gate. Primary difference:

$$G_{\text{recurrence}} = K_{\text{AR}} - K_{\text{internal}}.$$

Report full curves, not only maxima.

8 Corrected Stage-1 results

The first learned comparison fixes the passing R0-v2 architecture (L2/W64/H4), uses 1,000 updates per cell, trains only at $K \in \{1, 2, 3\}$, and evaluates 64 held-out-pair chains at each $K \in \{1, 2, 3, 4, 6, 8\}$ for seeds 11, 23, and 37. Competence requires at least 95% final and termination accuracy for every seed. Table 1 reports only measured points.

Condition	Supervision	Three-seed frontier	$P_{\text{acquire}}(K = 4)$
O0	final answer	3	0/3
O1	free proposition trace	3	0/3
O2	structure+final, no states	none	0/3
O3	full structured trace	3	0/3

Table 1: Leakage-free Stage-1 frontiers. “None” means that O2 fails the gate even in range; it does not mean that the model produces no correct tokens.

O0 is 100% for all seeds at $K = 1, 2, 3$, then falls at $K = 4$ to 0%, 1.6%, and 4.7%. O1 is likewise perfect in range but reaches only 1.6%, 1.6%, and 0% at $K = 4$. Its transition accuracy at that boundary is 32.0%, 32.0%, and 23.4%; at $K = 6, 8$ it falls to roughly 10–20%. Termination, perfect in range, becomes seed- and depth-dependent out of range. Free generation therefore adds writable tokens but does not acquire a reusable transition in this regime.

O2 separates syntax from semantics. Across depths its seed-level aggregate final accuracy is 9.1%, 25.0%, and 3.4%; transition accuracy is 1.3–4.6%, and no exact trajectory occurs, although termination can remain high. The model learns much of the requested output grammar without the missing state labels. O3 restores perfect in-range trajectories, yet at $K = 4$ final accuracy is only 1.6%, 0%, and 0%; deeper values remain near chance. Richer derivation supervision improves interpolation but does not create extrapolative recurrence.

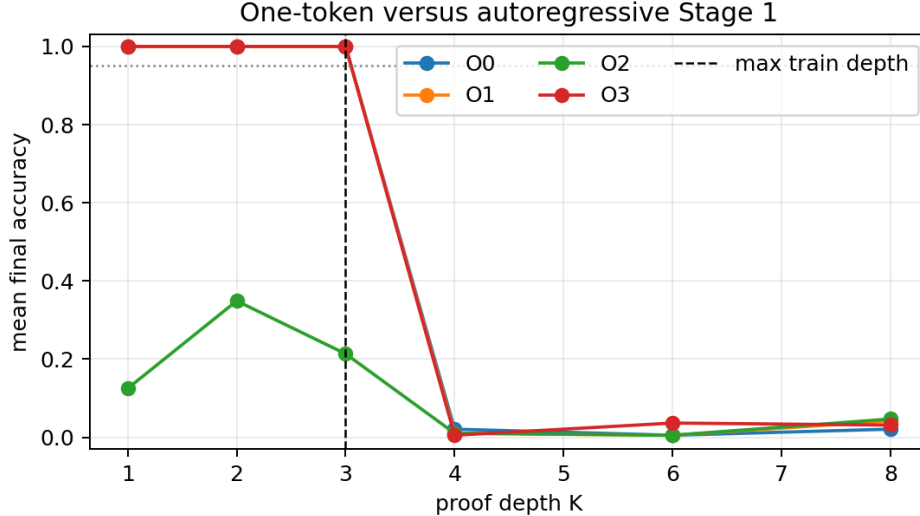


Figure 1: Mean three-seed final accuracy for the corrected Stage-1 comparison. The dashed line marks the maximum training depth. The sharp shared boundary, rather than gradual accumulated error, is inconsistent with a measured closure signature.

Consequently,

$$K_{\text{internal}} = K_{\text{AR}} = K_{\text{O3}} = 3, \quad G_{\text{recurrence}} = 0$$

over the tested architecture, supervision, and depths. This is a finite empirical statement. It neither proves that autoregression cannot expand the frontier under another curriculum/architecture nor licenses infinite-closure claims.

9 Closure signatures

Train on $K \leq 2$ or 3 and test much larger K . Evidence for reusable recurrence strengthens when:

1. accuracy remains high far beyond K_{train} ;
 2. generated step count scales with true proof depth;
 3. the same local transition is used at early and late steps;
 4. state representations align by proof role rather than absolute generation index;
 5. failure depends mainly on accumulated local error or context budget rather than a sharp learned maximum length.
- Do not claim literal $K = \infty$ closure from finite tests.

10 Finite-context versus idealized closure

Let C_{max} be context capacity. Even a correct recurrence eventually fails if its serialized trajectory exceeds available memory. Report $K_{\text{AR}}(M, C_{\text{max}})$ and distinguish procedure failure from context overflow.

11 Acquisition probability

For architecture M and training distribution $\mathcal{D}$ define

$$P_{\text{acquire}}^{\text{recurrence}} = P_s[A(K_{\text{test}} \gg K_{\text{train}}) \geq \gamma].$$

Distinguish recurrence representable but rarely acquired, recurrence reliably acquired, and local rule acquired but recurrence not acquired.

12 Architecture grid

Start with

$$L \in \{1, 2, 3, 4\}, \quad W \in \{16, 32, 64\}, \quad H \in \{1, 2, 4\},$$

and training budgets $T \in \{1, 2, 4\} \times$. This diagnostic grid, four output conditions, and three model seeds enumerates 1,296 possible phase cells, not completed evidence. Stage 1 runs only the 12-cell passing R0 architecture slice. Deeper $L \in \{6, 8\}$ and wider $W = 128$ cells remain conditional expansions. The next justified step is a narrow boundary/curriculum test, followed by shuffled rules, distractors, and branching only if recurrence becomes competent.

13 Mechanistic measurements

At every generation step record next-token distribution, correct-next-state rank/margin, Q/K/V, attention role mass, SA/FF residual increments, representation similarity across recurrence steps, and attention to original rules versus generated state.

For correct next state s_{t+1}^* define

$$m_t = z_t(s_{t+1}^*) - \max_{y \neq s_{t+1}^*} z_t(y).$$

Test whether m_t degrades with absolute step index after controlling local-rule difficulty.

14 Causal tests

For competent recurrent models:

- patch a same-role state from another proof depth;
 - replace an intermediate proposition with correct/incorrect alternatives;
 - mask previous proof states while preserving original rules;
 - mask original rules after they are supposedly encoded into generated state;
 - ablate heads/FF components by recurrence step;
 - compare same-role transitions across unseen proposition identities.
- A genuine recurrence should be more invariant to absolute step number than a memorized fixed-length trajectory.

15 Error accumulation and termination

Estimate conditional transition accuracy

$$p_t = P(s_{t+1} \text{ correct} \mid s_{\leq t} \text{ correct}).$$

Compare whole-proof accuracy with $\prod_t p_t$ as a simple independent-error baseline. Analyze premature answer, failure to terminate, trajectory drift, wrong applicable rule, and context overflow separately.

Under the deliberately restrictive baseline that transitions are independent conditional on a correct prefix and $p_t = p$, whole-proof success is p^K . The experiment must compare observed success against $\prod_t p_t$, because attention to the full generated history permits correlated errors and self-repair. A positive or negative deviation is therefore a diagnostic of dependence, not automatically evidence for or against recurrence.

16 CPU-validated serialization frontier

The prompt token count in the pure ordered chain is $4K + 6$ under the whitespace audit tokenizer. O0 adds one target token, O1 adds $K + 1$, and O2/O3 add $2K + 5$. At $K = 64$, the resulting audited totals are 263, 327, and 395 tokens, respectively. These values provide exact context-budget denominators for the present serialization; tokenizer-specific model runs must also report their true subword lengths.

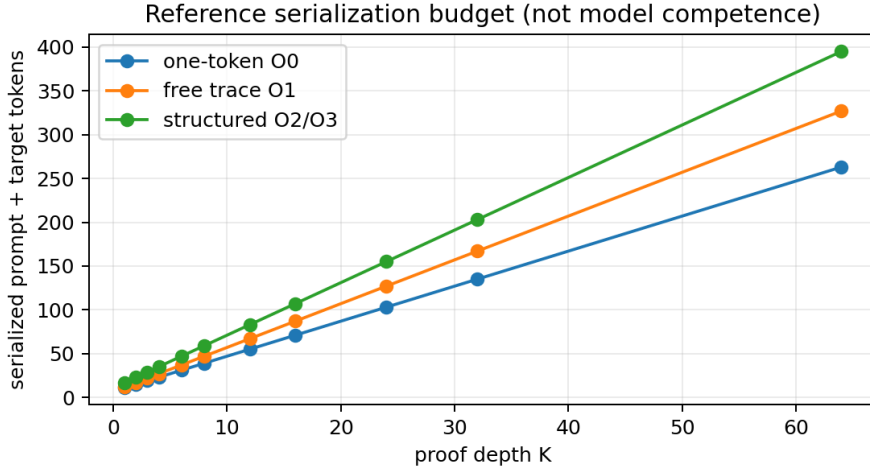


Figure 2: Prompt-plus-target length for the validated reference serialization. This is a dataset geometry result, not a learned competence frontier.

The learned Stage-1 results above report accuracy, measured frontiers, transition accuracy, and termination. Role invariance and causal recurrence tests remain blocked because no condition is recurrently competent beyond the training range. Exact oracle performance is never entered into a model-results table.

17 ICL extension

After base recurrence is understood, compare rules learned in weights against novel implication graphs supplied entirely in context with held-out symbols. If the latter succeeds at long depth, the clean interpretation is:

context supplies instance-specific facts/rules + weights supply recurrence procedure.

Do not add this extension before a stable recurrence regime exists.

18 Relation to Papers 0.7 and 0.9

Paper 0.7 measures one-pass formal computation. Paper 0.85 adds self-generated intermediate state. Paper 0.9 adds deterministic external operations and an explicit loop. The required ladder is

one-token $\rightarrow$ free multi-token $\rightarrow$ structured scratchpad $\rightarrow$ tool/harness.

Only this can distinguish class expansion from improved learnability/reliability.

19 Expected contribution

The strongest positive result would show tiny Transformers trained only on short proofs acquiring a reusable autoregressive transition that extrapolates to much longer chains while one-token performance remains bounded. The strongest negative result would show reliable local implication but bounded or seed-fragile recurrent trajectories. Either outcome locates a computational boundary between fixed-depth composition and explicit external iteration.

20 Reproducibility and evidence status

The executable components are:

- generator/evaluator: `src/cl/semantic/autoregressive_proofs.py`;
- phase materialization: `src/cl/experiments/paper085_phase.py`;
- serialization analysis: `src/cl/experiments/paper085_analyze.py`;
- configuration: `configs/paper085/phase_v1.json`.
- R0-v2 gate: `paper085_r0_gate_v2.py` with `r0_gate_v2.json`;
- recurrence runner: `paper085_recurrence_stage1.py`;
- recurrence analysis: `paper085_recurrence_analyze.py`;
- recurrence configuration: `recurrence_stage1_v2.json`.

Machine-readable artifacts are under `docs/papers/paper0_85/results`. The broad phase manifest explicitly labels its 11,200 exact traces as reference results rather than learned evidence. Learned R0-v1, R0-v2, and corrected Stage1-v2 occupy isolated directories; the Stage1-v1 manifest is quarantined because of query-target leakage. Tracked dense checkpoints are locally reproducible and may remain ignored. Planned long-depth, context-limit, role-invariance, causal, shuffled-rule, distractor, branching, and ICL experiments are not stated as findings.

References

- [1] Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of Thought Empowers Transformers to Solve Inherently Serial Problems. *arXiv:2402.12875*, 2024.
- [2] N. Joshi et al. A Theory of Learning with Autoregressive Chain of Thought. *COLT*, 2025.
- [3] A. A. Bavandpour et al. Lower Bounds for Chain-of-Thought Reasoning in Hard-Attention Transformers. *ICML*, 2025.